

rpm包如何构建

1. RPM(RedHat Packager Manager) Basics

1.1 rpm和srpm

rpm即软件包的一种管理方式。Debian Linux社群用的是dpkg。

binary rpm类似与安装包，可以直接将其安装，其中包含的软件是经过编译的(相对于srpm)。

格式为：unzip-6.0-20.el7.centos.es.x86_64.rpm (n <name> : unzip, v <version> :6.0, r <release> :20.el7.centos.es,a<arch>:x86_64)

安装时会将软件信息写入到RPM数据库(/var/lib/rpm)，便于查询。

```
1 rpm -i /tmp/unzip-6.0-41.el8.es.rpm //安装binary rpm，即为将包含的文件释放到一些指定目录
2 rpm -qa //查看安装的所有软件
3 rpm -qi unzip //查看某个安装的软件详细信息
4 rpm -q --queryformat "%{NAME}\n" unzip //查看某个安装的软件的特定位段
5 rpm -q --changelog unzip //查看changelog
```

优点：便于查询，升级和卸载，无需自己手动编译，安装方便

缺点1：安装的环境需要与打包时的环境需求一致或相当

srpm的存在可以使得包重新编译，srpm即为Source RPM，内容没有经过编译，提供原始代码，比如说: unzip-6.0-20.el7.src.rpm，openssl-1.1.1g-15.el8_3.src.rpm(nvr可以表示)，一个srpm包可以编译出来多个binary rpm包。

```
1 [root@localhost openssl]# rpm -q --queryformat "%{NAME}\n" -p ~/xorg-x11-xauth-1.0.9-12.el8.src.rpm
2 xorg-x11-xauth
3
4 [root@localhost openssl]# rpm -q --changelog -p ~/xorg-x11-xauth-1.0.9-12.el8.src.rpm | head -3
5 * 3月 19 2018 Adam Jackson <ajax@redhat.com> - 1.0.9-12
6 - Nerf the python-related BuildRequires. They're only needed for 'make check',
7 and cmdtest is not yet a python3 package.
```

1.2 srpm解压（也称为安装）后的文件内容

查看下文件解压后的内容：

```
1 rpm -i --define '_topdir /root/build/openssl' ./openssl-1.1.1g-15.el8_3.src.rpm #使用此命令解压到指定文件夹
2 [root@localhost openssl]# tree
3 .
4 |--- SOURCES
5 | |--- ec_curve.c #源代码
6 | |--- ectest.c #源代码
7 | |--- hobble-openssl #源代码
8 | |--- make-dummy-cert
9 | |--- Makefile.certificate
10 | |--- openssl-1.1.0-issuer-hash.patch
11 | |--- openssl-1.1.1-alpn-cb.patch
12 | |--- openssl-1.1.1-apps-dgst.patch
13 | |--- openssl-1.1.1-arm-update.patch
14 | |--- openssl-1.1.1-build.patch
15 | |--- openssl-1.1.1-conf-paths.patch
16 | |--- openssl-1.1.1-CVE-2020-1971.patch
17 | |--- openssl-1.1.1-CVE-2021-3449.patch
18 | |--- openssl-1.1.1-CVE-2021-3450.patch
19 | |--- openssl-1.1.1-defaults.patch
20 | |--- openssl-1.1.1-ec-curves.patch
21 | |--- openssl-1.1.1-edk2-build.patch
22 | |--- openssl-1.1.1-evp-kdf.patch
23 | |--- openssl-1.1.1-explicit-params.patch
24 | |--- openssl-1.1.1-fips-crmg-test.patch
25 | |--- openssl-1.1.1-fips-curves.patch
26 | |--- openssl-1.1.1-fips-dh.patch
27 | |--- openssl-1.1.1-fips-drbg-selftest.patch
28 | |--- openssl-1.1.1-fips.patch
29 | |--- openssl-1.1.1-fips-post-rand.patch
30 | |--- openssl-1.1.1g-hobbled.tar.xz #源代码
31 | |--- openssl-1.1.1-ignore-bound.patch
32 | |--- openssl-1.1.1-intel-cet.patch
33 | |--- openssl-1.1.1-kdf-selftest.patch
34 | |--- openssl-1.1.1-krb5-kdf.patch
35 | |--- openssl-1.1.1-man-rename.patch
36 | |--- openssl-1.1.1-no-brainpool.patch
37 | |--- openssl-1.1.1-no-html.patch
38 | |--- openssl-1.1.1-no-weak-verify.patch
39 | |--- openssl-1.1.1-reneg-no-extms.patch
40 | |--- openssl-1.1.1-rewire-fips-drbg.patch
41 | |--- openssl-1.1.1-s390x-ecc.patch
42 | |--- openssl-1.1.1-s390x-update.patch
```

```

43 | |— openssl-1.1.1-seclevel.patch
44 | |— openssl-1.1.1-ssh-kdf.patch
45 | |— openssl-1.1.1-ssl3-keep-abi.patch
46 | |— openssl-1.1.1-system-cipherlist.patch
47 | |— openssl-1.1.1-ts-sha256-default.patch
48 | |— openssl-1.1.1-version-add-engines.patch
49 | |— openssl-1.1.1-version-override.patch
50 | |— openssl-1.1.1-weak-ciphers.patch    # 源代码内容修补
51 | |— opensslconf-new.h                # 源代码
52 | |— opensslconf-new-warning.h
53 | |— README.FIPS
54 | |— renew-dummy-cert
55 |— SPECS
56 |— openssl.spec                      # 配置文件

```

大概看下spec文件：

```

1  # basics
2  Summary: Utilities from the general purpose cryptography library with TLS implementation
3  Name: openssl
4  Version: 1.1.1g
5  Release: 15%{?dist}
6  Epoch: 1
7  License: OpenSSL and ASL 2.0
8  URL: http://www.openssl.org/
9
10 # source
11 Source: openssl-%{version}-hobbled.tar.xz
12 Source1: hobble-openssl
13 Source2: Makefile.certificate
14 ...
15
16 # patch
17 Patch1: openssl-1.1.1-build.patch
18 Patch2: openssl-1.1.1-defaults.patch
19 ...
20
21 # require
22 BuildRequires: gcc
23 BuildRequires: coreutils, perl-interpreter, sed, zlib-devel, /usr/bin/cmp
24 ...
25 Requires: coreutils
26 Requires: %{name}-libs%{?_isa} = %{epoch}:%{version}-%{release}
27
28 # build
29 %prep #编译前的预处理
30 ...
31
32 %build #build指令
33 ...
34 make all
35 ...
36
37 # install
38 %install
39 [ "$SRPM_BUILD_ROOT" != "/" ] && rm -rf $SRPM_BUILD_ROOT
40 # Install OpenSSL.
41 install -d $SRPM_BUILD_ROOT%{_bindir}%{_includedir}%{_libdir}%{_mandir}%{_libdir}/openssl,%{_pkgdocdir}
42 make DESTDIR=$SRPM_BUILD_ROOT install
43 ...
44
45 # 释出的多个rpm
46 %files
47 %{!?_licensedir:%global license %doc}
48 %license LICENSE
49 %doc FAQ NEWS README README.FIPS
50 %{_bindir}/make-dummy-cert
51 %{_bindir}/renew-dummy-cert
52 %{_bindir}/openssl
53 %{_mandir}/man1/*/*
54 %{_mandir}/man5/*/*
55 %{_mandir}/man7/*/*
56 %{_pkgdocdir}/Makefile.certificate
57 %exclude %{_mandir}/man1/*/*.*.pl*
58 %exclude %{_mandir}/man1*/c_rehash*
59 %exclude %{_mandir}/man1*/tsget*
60 %exclude %{_mandir}/man1*/openssl-tsget*
61
62 %files libs
63 %{!?_licensedir:%global license %doc}
64 %license LICENSE
65 %dir %{_sysconfdir}/pki/tls
66 %dir %{_sysconfdir}/pki/tls/certs

```

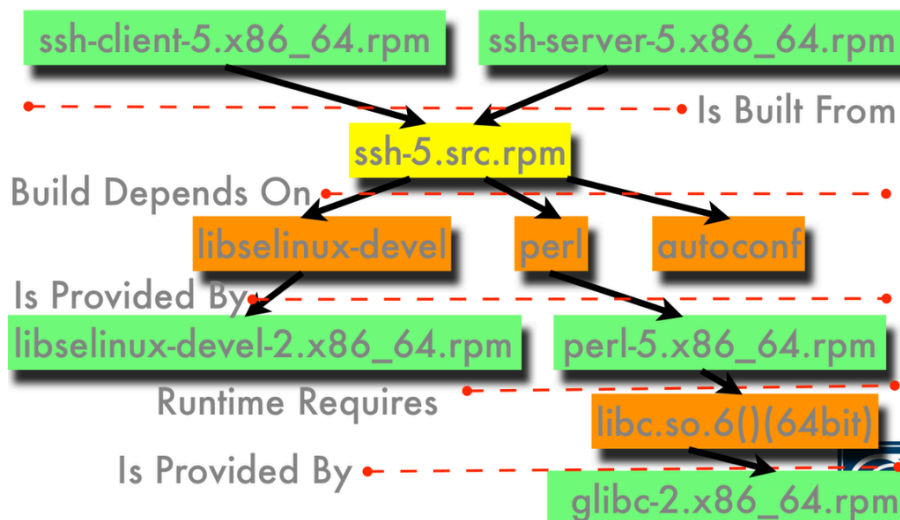
```

67 %dir %{_sysconfdir}/pki/tls/misc
68 %dir %{_sysconfdir}/pki/tls/private
69 %config(noreplace) %{_sysconfdir}/pki/tls/openssl.cnf
70 %config(noreplace) %{_sysconfdir}/pki/tls/ct_log_list.cnf
71 %attr(0755,root,root) %{_libdir}/libcrypto.so.%{version}
72 %attr(0755,root,root) %{_libdir}/libcrypto.so.%{soversion}
73 %attr(0755,root,root) %{_libdir}/libssl.so.%{version}
74 %attr(0755,root,root) %{_libdir}/libssl.so.%{soversion}
75 %attr(0644,root,root) %{_libdir}/libcrypto.so.*.hmac
76 %attr(0644,root,root) %{_libdir}/libssl.so.*.hmac
77 ...
78
79 # changelog
80 %changelog
81 * Mon Feb 25 2019 Jakub Martisko <jamartis@redhat.com> - 6.0-20
82 - Fix CVE-2018-18384
83   Resolves: CVE-2018-18384
84
85 * Tue Jan 09 2018 Jakub Martisko <jamartis@redhat.com> - 6.0-19
86 - rename patch unzip-6.0-nostrip.patch to unzip-6.0-configure.patch
87 - make linking flags configurable from the spec file
88 - set linking flags to use relro hardening
89   Related: #1531116
90 ...

```

- Summary: 本软件的主要说明
- Name: 本软件的软件名称
- Version: 本软件版本号
- Release: 本软件的打包次数说明, 发行版本. ×× 我们编译过程中, 需要做修改, 我们约定的规则是, Release: x%{?dist}_n, x为上游或(我们的)修改次数; _n用来判断我们有无修改. 第一个点 %{?dist} 会依据编译环境调整方便我们判断是要给 v6 或 之后的v6-next 使用. 第二个点 _n 是方便判读 是否经过我们的修改, 更好的追溯到上游。
- Epoch: 版本好回退修改时使用, 一般由大版本到小版本时, 增加Epoch(默认是0), 注意比较版本号的时候需要比较Epoch
- SourceN: 软件源代码的来源, 可能是url
- PatchN: 软件补丁
- Requires: 制作成rpm包安装时, 系统检查依赖的软件包
- BuildRequires: 构建该软件包的依赖
- %prep: 进行编译前的处理, 一般就是检查和补丁工作
- %build: 构建包的指令执行
- %install: 安装软件执行指令
- %files: 编译出来的binary rpm包名称及各个binary rpm包含的文件
- %changelog: 修改记录, ××我们项目中如果需要修改srpm并重新编译, 需要按照规则修改

1.3 dependencies



1.4 rpm build实例

```

1 rpmbuild -ba unzip.spec // 编译产生rpm和srpm文件
2 rpmbuild -bb unzip.spec // 编译仅产生rpm文件
3 rpmbuild -bs unzip.spec // 编译仅产生srpm文件

```

我们来执行ba操作

```

1 [root@localhost SPECS]# rpmbuild -ba --define '_topdir /root/build/unzip' ./unzip.spec
2 错误：构建依赖失败：
3     bzip2-devel 被 unzip-6.0-20.el8.x86_64 需要

```

这里我们需要首先安装bzip2-devel

```

1 yum install bzip2-devel -y # 还要安装 make, gcc等

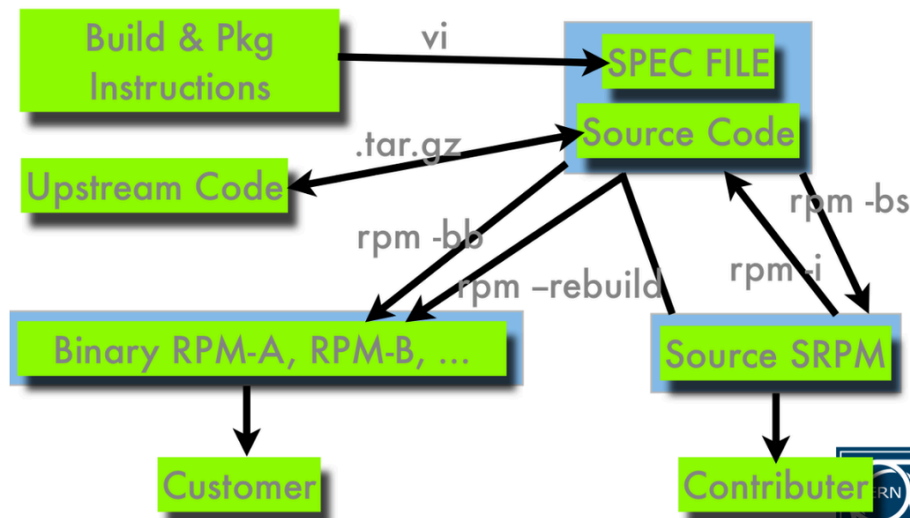
```

最后就会生成了一下文件：

```

1 # 编译unzip输出
2 [root@localhost unzip]# tree RPMS/ SRPMS/
3 RPMS/
4   └─x86_64
5       ├──unzip-6.0-20.el8.x86_64.rpm
6       ├──unzip-debuginfo-6.0-20.el8.x86_64.rpm
7       └──unzip-debugsource-6.0-20.el8.x86_64.rpm
8 SRPMS/
9   └─unzip-6.0-20.el8.src.rpm
10
11 #编译openssl输出的binary rpm
12 openssl-1.1.1g-15.el8.x86_64.rpm
13 openssl-lib-1.1.1g-15.el8.x86_64.rpm
14 openssl-devel-1.1.1g-15.el8.x86_64.rpm
15 openssl-static-1.1.1g-15.el8.x86_64.rpm
16 openssl-perl-1.1.1g-15.el8.x86_64.rpm
17 openssl-debugsource-1.1.1g-15.el8.x86_64.rpm
18 openssl-debuginfo-1.1.1g-15.el8.x86_64.rpm
19 openssl-lib-1.1.1g-15.el8.x86_64.rpm

```



缺点2：构建和安装依赖需要先安装依赖的包

1.5 yum

可以解决安装时软件依赖的问题

```

1 yum install ./xxx.rpm

```

2. MOCK

主要用于在chroot 环境下，提供一个干净的环境来编译RPM，使得本地环境没有依赖，同时编译过程中会自动在chroot环境去安装依赖的软件包，mock的本质就是在chroot环境中执行rpmbuild

2.1 如何使用

需要使用srpm为前提：

```

1 mock -r ./centos-8-x86_64.cfg --rebuild ~/build/unzip/SRPMS/unzip-6.0-20.el8.src.rpm

```

或者

```

1 mock -r ./centos-8-x86_64.cfg --init
2 mock -r ./centos-8-x86_64.cfg --rebuild ~/build/unzip/SRPMS/unzip-6.0-20.el8.src.rpm
3 mock -r ./centos-8-x86_64.cfg --clean

```

看下cfg文件：

```
1 # meta包
2 onfig_opts['chroot_setup_cmd'] = 'install tar gcc-c++ redhat-rpm-config redhat-release which xz sed make bzip2 gzip gcc coreutils unzip shadow-utils diffutils cpio bash gawk rpm-build info patch util-linux findutils grep'
3 ...
4
5 config_opts['dist'] = 'el8'
6 config_opts['releasever'] = '8'
7 config_opts['package_manager'] = 'dnf'
8 ...
9
10 # repo
11 [baseos]
12 name=CentOS-$releasever - Base
13 mirrorlist=http://mirrorlist.centos.org/?release=8&arch=$basearch&repo=BaseOS&infra=$infra
14 failovermethod=priority
15 gpgkey=file:///usr/share/distribution-gpg-keys/centos/RPM-GPG-KEY-CentOS-Official
16 gpgcheck=1
17 skip_if_unavailable=False
18 [appstream] ...
19 [powertools] ...
20 [devel] ...
```

2.2 mock执行时构建步骤

- 初始化chroot环境，创建文件夹（/var/lib/mock/centos-8-x86_64/root）
- 该环境中填充一些文件夹（/etc/, /bin/, /dev/null/）
- 拷贝srpm到chroot
- 安装meta包
- 从配置文件的仓库拉取并安装要依赖的包（配置文件 # repo）
- 创建用户
- build (rpmbuild --rebuild)

2.3 mock caching

- mock提供了很多插件用来做cache（ccache，root_cache，yum_cache）（/var/cache/mock）
- local repo // cfg文件修改源指向本地

3. KOJI

3.1 koji结构和功能

koji是rpm包的构建和管理系统，且可以制定多台builder构建

Koji

Recent Builds

ID

▼

NVR

Built by

Finished

State

5709

ovn-21.03.0-2.fc35.Ses

admin

2021-04-23 12:18:23

5708

coaster-0.1.Ses.1

admin

2021-04-16 19:44:47

5707

escore-release-6.1.1.Ses.1

admin

2021-04-13 21:06:28

5706

lua5.2-2.0-32.2.Ses

shawn.wang

2021-04-13 17:34:15

5705

lua5.2-2.0-4-3.at7.Ses

shawn.wang

2021-04-13 16:08:59

5704

sysbench-1.0.17-2.at7.Ses

shawn.wang

2021-04-13 22:16:57

5703

kernel-3.10.0-693.11.1.at7.el8.14

shawn.wang

2021-04-08 13:39:59

5702

kernel-3.10.0-514.16.1.at7.6

shawn.wang

2021-04-08 13:38:36

5701

etherinet-req-affinity-1.0-6.Ses.4

admin

2021-04-08 13:37:13

5700

netstat-rs4-1.4.10-20.fc34.Ses

shawn.wang

2021-04-01 17:57:46

Recent Builds

ID

▼

NVR

Built by

Finished

State

5709

ovn-21.03.0-2.fc35.Ses

admin

2021-04-23 12:18:23

5708

coaster-0.1.Ses.1

admin

2021-04-16 19:44:47

5707

escore-release-6.1.1.Ses.1

admin

2021-04-13 21:06:28

5706

lua5.2-2.0-32.2.Ses

shawn.wang

2021-04-13 17:34:15

5705

lua5.2-2.0-4-3.at7.Ses

shawn.wang

2021-04-13 16:08:59

5704

sysbench-1.0.17-2.at7.Ses

shawn.wang

2021-04-13 22:16:57

5703

kernel-3.10.0-693.11.1.at7.el8.14

shawn.wang

2021-04-08 13:39:59

5702

kernel-3.10.0-514.16.1.at7.6

shawn.wang

2021-04-08 13:38:36

5701

etherinet-req-affinity-1.0-6.Ses.4

admin

2021-04-08 13:37:13

5700

netstat-rs4-1.4.10-20.fc34.Ses

shawn.wang

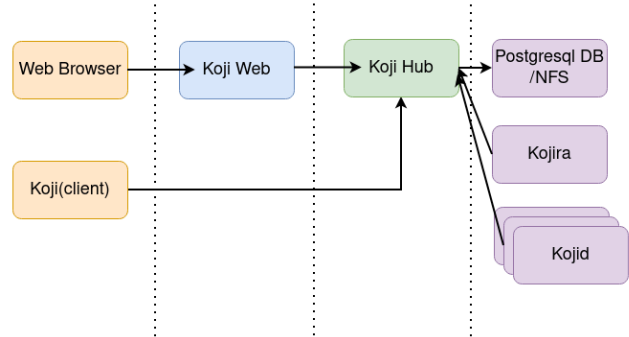
2021-04-01 17:57:46

Search

Packages

Search

koji的结构图：



- koji-hub：这个其实是整个koji架构的中心，负责着整个系统的维护和各个组件之间的数据交换。是一个xml-rpc的server，通过mod_wsgi模块挂载在apache上提供服务。接受client和web到来的请求，然后驱使其他组件完成构建工作，同时将数据存放到数据库中。
访问 /kojihub会定位到/koji-hub/kojixmlrpc.py文件来解析，只接收POST 请求
- kojid：部署在要完成构建工作的相应机器上，可以部署到多个机器。主要负责从Koji-hub请求到任务进行处理，一般来讲就是构建任务，××我们分别在构建x86_64体系和构建aarch64两台机器部署了 kojid
- koji-web：主要提供给使用浏览器的一些接口，然后从上游（hub）获取数据传给浏览器，浏览器访问<http://mirror.easystack.cn/koji/xxx>，根据apache的kojiweb.conf配置，就会访问到web组件的代码文件 wsgi_publisher.py
- koji（client）：命令行的客户端，输入指令与hub交互，执行一些指令（添加用户，提交build请求等等）
- kojira：主要负责和仓库建立，仓库更新等相关事宜。这里仓库不是指数据库，是指rpm repo。应该是只有一个实例在运行。××我们没有使用kojira
- Postgresql DB: 数据库，只有hub和他建立连接，并存取一些关系的数据，比如说账号信息，构建信息，repo的信息等等。××我们Postgresql服务和hub同一台机器

3.2 koji的一些概念

package：指包的名字，除去version,release等，单纯指name，例如 unzip-6.0-41.el8.es.src.rpm，package是指unzip

build：我们所说的koji build是指构建这个动作，同时我们也会说产生nvr包也称为build，产生nvr的后果就是这个build包名字在koji系统中只能存在一个，不能重名。

tag：build-tag和destination-tag，tag可以继承。build-tag是指构建时环境准备及**仓库的指定**。destination-tag即我们最终为build出来的包指定tag，可以理解作为作为一个管理多个build包的标识。tag之间是可以继承的，继承时会获得父亲的所有配置

target：(build target),关联一个build-tag及destination-tag，但是tag可以关联多个target，当我们build的时候只需要制定target就可以以build-tag来打包，用destination-tag来管理包

3.3 koji build

3.3.1 build的流程

前置条件

- 创建tag

```
1 koji add-tag dist-foo
2 koji add-tag --parent dist-foo --arches "x86_64" dist-foo-build
```

- 添加构建仓库

```
1 koji add-external-repo -t dist-foo-build base-external-repo https://mirrors.aliyun.com/centos/8/BaseOS/$arch/os/
2 koji add-external-repo -t dist-foo-build appstream-external-repo https://mirrors.aliyun.com/centos/8/AppStream/$arch/os/
3 koji add-external-repo -t dist-foo-build epel-external-repo https://mirrors.aliyun.com/epel/8/Everything/$arch/
```

- 加入构建组

```
1 koji add-group dist-foo-build build
2 koji add-group dist-foo-build srpm-build
```

- 添加meta包，这里是从mock配置文件拷贝，最终从仓库添加

```
1 koji add-group-pkg dist-foo-build build tar gcc-c++ redhat-rpm-config redhat-release which xz sed make bzip2 gzip gcc coreutils unzip shadow-utils diffutils cpio bash gawk rpm-build info patch util-linux findutils grep
2
3 koji add-group-pkg dist-foo-build srpm-build tar gcc-c++ redhat-rpm-config redhat-release which xz sed make bzip2 gzip gcc coreutils unzip shadow-utils diffutils cpio bash gawk rpm-build info patch util-linux findutils grep
```

- 添加相应的pkg到指定tag，表示可以在该tag下编译这个pkg

```
1 koji add-pkg --owner kojiadmin dist-foo unzip
```

build

- build包

```
1 koji build dist-foo /opt/unzip-6.0-20.el7.src.rpm
```

注：

- 每次koji build使用的仓库是由build-tag制定，为mock生成一个cfg文件，然后调用mock
- ××我们项目可以直接做build操作，前置条件的准备已经在搭建环境时准备好了，我们使用的target一般是ecs，看下相关信息：

```
1 // 查询target
2 [zhangdixin@se129 ~]$ koji list-targets --name=ecs
3 Name           Buildroot           Destination
4 -----
5 ecs             ecs-buildd          ecs-pending
6
7 // 查询下tag的信息
8 [zhangdixin@se129 ~]$ koji taginfo ecs-buildd
9 Tag: ecs-buildd [95]
10 Arches: x86_64 aarch64
11 Groups: base, build, livecd-build, srpm-build
12 Tag options:
13   mock.package_manager : 'yum'
14 This tag is a buildroot for one or more targets
15 Current repo: repo#1691: 2021-04-13 21:09:16.503853
16 Targets that build from this tag:
17   escl77-extras2
18   ecs
```

```
19 External repos:
20 1 i386-i386 (http://mirror.easystack.cn/kojifiles/repos/i386/latest/i386/)
21 2 i386-x86_64 (http://mirror.easystack.cn/kojifiles/repos/i386/latest/x86_64/)
22 5 centos7-os (http://mirror.easystack.cn/mirrors/centos/7/os/$arch/)
23 10 centos7-extras (http://escore.escore@mirror.easystack.cn/mirrors/centos/7/extras/$arch/)
24 15 epel7 (http://mirror.easystack.cn/mirrors/epel/7/$arch/)
25 20 sclo7 (http://mirror.easystack.cn/mirrors/centos/7/sclo/$arch/rh/)
26 Inheritance:
27 1 .... ecs-all [110]
28 2 .... ecs-extras [120]
```

- ×× 我们项目中build过程是先skip-tag，后边发布时再打上tag

```
1 koji build ecs --skip-tag ovn-21.03.0-2.fc35.5es.src.rpm
```

3.3.2 build选项

build可以添加选项:

--skip-tag: 编译过程不加destination-tag，会产生NVR，可做临时检查使用，之后决定发布时再tag build（一旦生成NVR，不可以重新编译）

--scratch: 只产生rpm包，不会产生NVR，没有打tag，我们可以使用“koji import --create-build xxx.rpm”指令来产生NVR，然后再使用tag build上tag或者使用“koji call mergerScratch build的taskid”关联到存在的build上

3.3.3 跟踪一下

以“koji build dist-foo /opt/unzip-6.0-20.el7.src.rpm”为开始

```
python
-----#koji(cli)-----|
|# cli/koji.py __main__|
|session = koji.ClientSession(options.server, session_opts)|
|locals()[command].__call__(options, session, args)|
|
|
|
|
|-----#kojihub-----|
|# cli/commands.py handle_build| |---->| # kojihub.py get_build_target
|session.getBuildTarget(target)| |    | return # 查询数据库，返回字段
|                                     | |    | (
|                                     | |    |     ('build_target.id', 'id'),
|                                     | |    |     ('build_tag', 'build_tag'),
|                                     | |    |     ('dest_tag', 'dest_tag'),
|                                     | |    |     ('build_target.name', 'name'),
|                                     | |    |     ('tag1.name', 'build_tag_name'),
|                                     | |    |     ('tag2.name', 'dest_tag_name'),
|                                     | |    | )
|
|session.uploadWrapper # 上传包| |
|session.build(source, target, opts, priority)| ---->| #kojihub.py build
|-----| |make_task('build', [src, target, opts], **taskOpts)
|                                     | |
|                                     | |#kojihub.py make_task
|                                     | |get_channel_id
|                                     | |xmlrpcplus.dumps # encode xmlrpc request
|                                     | |
|                                     | |plugin.run_callbacks('preTaskStateChange')
|                                     | |InsertProcessor('task', data=idata) # insert db,get task id
|                                     | |plugin.run_callbacks('postTaskStateChange')
|                                     | |
|-----| |
|
|-----#kojid-----| |
|#kojid.py __main__| |
|koji.ClientSession(options.server, session_opts)| |
|koji.daemonize()| |
|main(options, session)| |
|
| |
| |
| |#kojid.py main(options, session)| |
|tm = TaskManager(options, session)| |
|tm.findHandlers(globals()) # 注册handler| |
|
| |
| |while 1:| |
| |    tm.updateBuildroots() # 更新构建环境| |
| |    tm.updateTasks() # 清理task或者唤醒task| |
| |    tm.getNextTask()| |
| |
| |
| |#daemon.py TaskManager.getNextTask| |
|updateHost(self.task_load, self.ready)| ---->| #kojihub.py updateHost 更新这个host的task负载和状态
|getLoadData() # 从hub获取tasks| -----| ---->| #kojihub.py get_active_tasks
|takeTask()| |
| |
| |
| |#daemon.py TaskManager.takeTask(task)| |
|getTaskInfo()| ---->| #kojihub.py getTaskInfo Return information about the task
|openTask(task['id'])| ---->| #kojihub.py Task.open
|forkTask(handler)| |-----|
|
| |
| |
| |#daemon.py TaskManager.forkTask| |-----|
| |fork子进程| |父进程
|runTask(handler)| |返回，继续getNextTask
|
| |
| |...
| |#daemon.py TaskManager.runTask| |#daemon.py TaskManager.runTask
|handler.run()| |handler.run()
|
| |
| |#kojid.py BuildTask.handler| |#kojid.py BuildArchTask.handler
|session.getTaskInfo(self.id)| |broot = BuildRoot() #buildroot 初始化
|session.getBuildTarget| |broot.init() # 初始化mock环境
|
| |broot.build(fn, arch) # 做mock编译
|getSRPM| |uploadFile # 上传文件
|
| |closeTask(handler.id, response)
|get_header_fields # 获取srpm的name,version,release
|session.getPackageConfig(dest_tag) # 获取dest_tag的信息,检查
|
|initBuild # get buildid
```



```
|runBuilds # block |
|
| #kojid.py BuildTask.runBuilds(srpm, build_tag) |
| subtask # make buildArch task |
| wait # 获取到编译完成的rpms |
|
|completeBuild |
| import_build |
| build_notification |
|tagBuild # make tagBuild task |
|-----|
|
|
```